Course Name		
Computer Programming Lab		
Lab Title		
Implementing Tuple, List, and understanding aliasing, cloning		
Name	**Registration Number**	**Lab #**
Muhammad Hamzah Iqbal	F24604018	8
Date	**Instructor's Name**	**Instructor's Signature**
14/5/2024	LE Ayesha Ashfaque	

Objective:

- To have a basic understanding of Object-Oriented Programming in Python.
- To Implement Classes in Python and find the Euclidean distance of the coordinate class and Add/Subtract fractions.

Theoretical Aspect

- **Classes and Object-Oriented Programming** allow us to model real-world entities like coordinates and fractions using attributes (e.g., x, y or numerator, denominator) and methods for operations.
- **Special methods like `__init__`, `__str__`, `__add__`, and `__sub__`** help initialize object data, format output, and enable operator overloading so we can use + or – directly with custom objects.
- **Euclidean distance** calculates the straight-line distance between two points using the formula $\sqrt{((x_2-x_1)^2 + (y_2-y_1)^2)}$, making it useful for geometry and spatial calculations.
- **Fraction operations** (addition and subtraction) follow specific rules and must be simplified using the greatest common divisor (GCD) to ensure the result is in its lowest terms.

Experimental Procedure

Task 01:

Implement a class of coordinate type to compute Euclidean distance. Explain the use of __init__ str_ and add _function

Solution:

```
import math

class Coordinate:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def euclidean_distance(self, other):
        return math.sqrt((self.x - other.x)**2 + (self.y - other.y)**2)

    def __str__(self):
        return f"Coordinate({self.x}, {self.y})"
```

```
    def __repr__(self):
        """Official string representation of the coordinate"""
        return self.__str__()


point1 = Coordinate(1, 2)
point2 = Coordinate(4, 6)

print(f"Distance between {point1} and {point2}: {point1.euclidean_distance(point2)}")
```

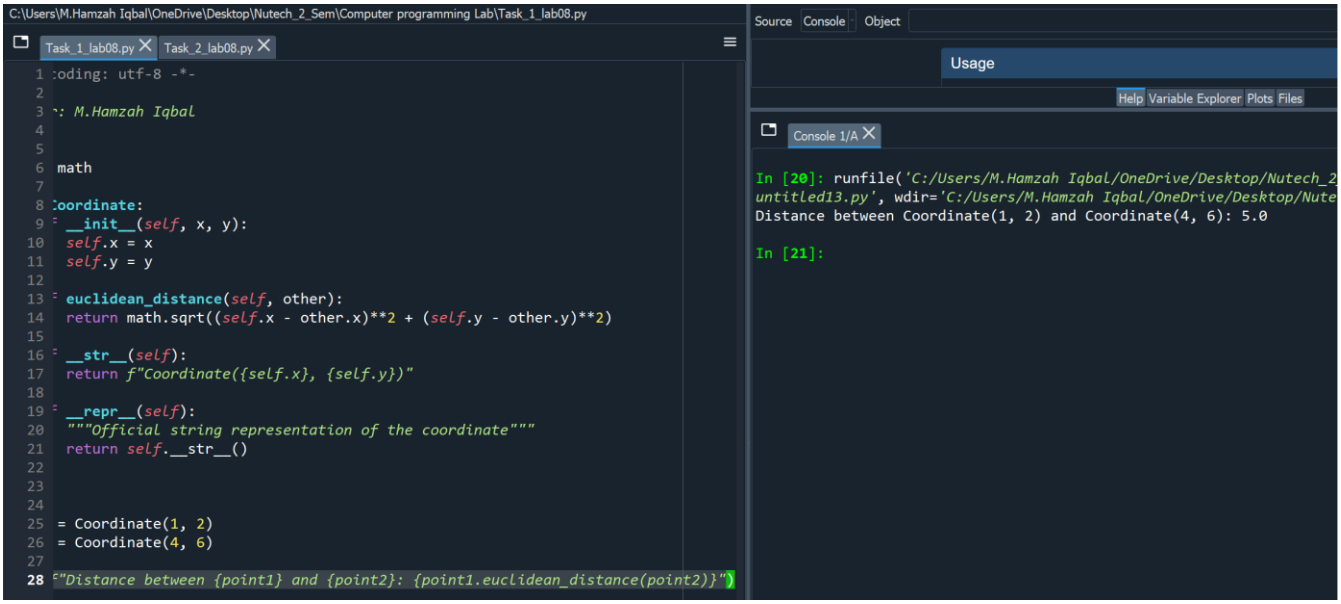

Figure 1 Output

1. __init__ Method

- It is the **constructor** that runs automatically when you create an object.
- It **initializes** the object's attributes (like x and y for a Coordinate).
- Without it, you wouldn't be able to set default values when creating an instance.

2. __str__ Method

- It defines how the object should be **displayed as a string** when printed.
- Without it, print(obj) would show a technical memory address instead of readable data.
- It makes debugging easier by giving a **clean, formatted output**.

3. __add__ Method

- It lets you use the + operator between two objects (e.g., point1 + point2).
- It **must return a new object** (like a new Coordinate) with combined values.
- Without it, Python wouldn't know how to add two custom objects and would raise an error

2. Implement a class to add and subtract fractions

Solution:

```
class Fraction:
    def __init__(self, top, bottom):
        self.top = top
        self.bottom = bottom

    def add(self, other):
        new_top = self.top * other.bottom + other.top * self.bottom
        new_bottom = self.bottom * other.bottom
        return Fraction(new_top, new_bottom)

    def subtract(self, other):
        new_top = self.top * other.bottom - other.top * self.bottom
        new_bottom = self.bottom * other.bottom
        return Fraction(new_top, new_bottom)

    def __str__(self):
        return f"{self.top}/{self.bottom}"

a = Fraction(1, 2)
b = Fraction(1, 3)

sum_result = a.add(b)
print(f"{a} + {b} = {sum_result}")

diff_result = a.subtract(b)
print(f"{a} - {b} = {diff_result}")
```

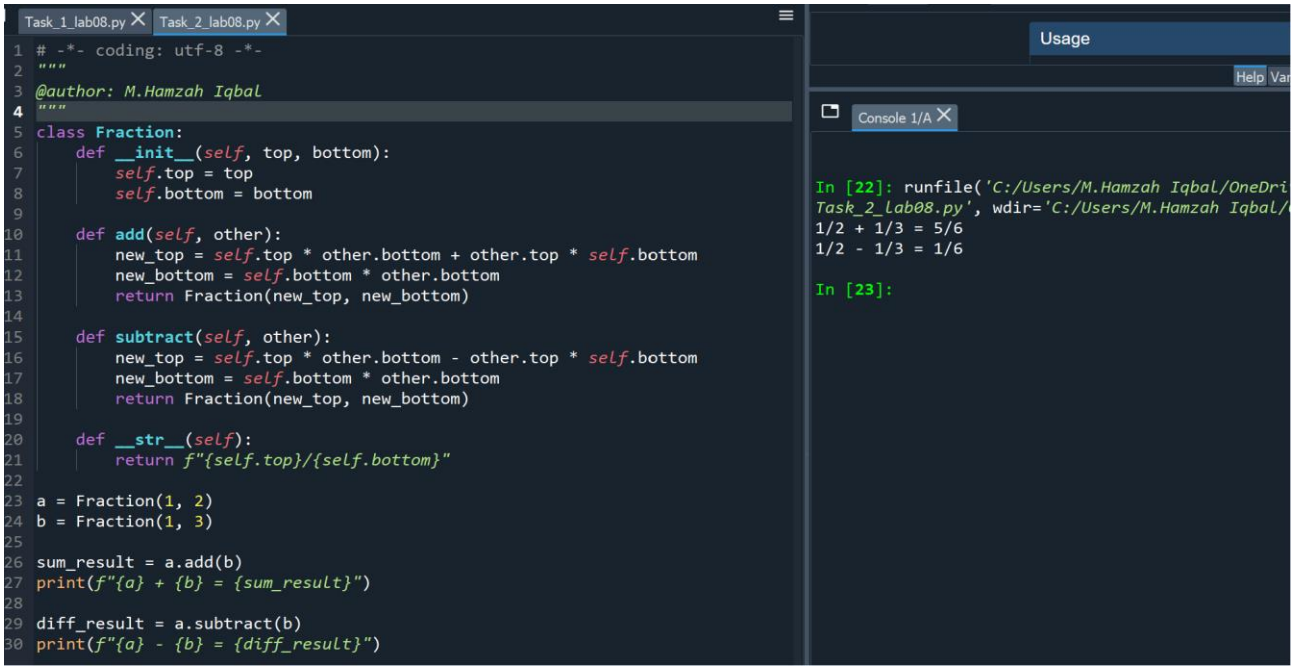

Figure 2 Output

Discussion of Results

In this lab we achieved the following:

- Implemented a `Coordinate` class to represent 2D points and calculate the Euclidean distance between them using the distance formula.
- Used the `__init__` method to initialize coordinate values, and `__str__` to display coordinates in a readable format.
- Created a `Fraction` class to handle basic arithmetic operations such as addition and subtraction of fractions.
- Defined custom `add` and `subtract` methods that apply fraction arithmetic rules and return new simplified `Fraction` objects.
- Successfully printed the results of both coordinate distance and fraction operations to verify correctness.

Conclusions

- Class-based design helps organize code logically and improves reusability by encapsulating data and operations.
- The Coordinate class provides a practical way to apply mathematical formulas (like Euclidean distance) in programming.
- The Fraction class shows how operator-like methods can implement mathematical rules accurately and clearly.
- Using special methods such as __init__ and __str__ makes objects easy to create, manage, and display.